Research Pipeline: Data Loading, Sampling, Preprocessing, and LIME Attribution

This notebook loads SNLI dataset, samples subsets, preprocesses them for attribution, and runs LIME explanations on SNLI examples.

Done for 300 samples of snli without stop words

PART 1: Setup environment and data

Import necessary libraries

In [1]:
```
import os
import json
import time
import random
import torch
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
import pandas as pd
from datasets import load_dataset, load_from_disk
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from lime.lime_text import LimeTextExplainer
from tqdm import tqdm
import nltk
from nltk.corpus import stopwords
try:
    stopwords.words("english")
except LookupError:
    nltk.download("stopwords")
import warnings
import datetime
import subprocess
from sklearn.metrics import confusion_matrix
import argparse

# Silence noisy logs
warnings.filterwarnings('ignore')
import transformers
transformers.logging.set_verbosity_error()
```

In [2]:
```
# === Labels (single source of truth) ===
# SNLI dataset ground-truth indices:
#   0 = entailment, 1 = neutral, 2 = contradiction
SNLI_LABELS = ['entailment', 'neutral', 'contradiction']
```

```
# roberta-large-mnli logits/probability indices:
#   0 = contradiction, 1 = neutral, 2 = entailment
MNLI_LABELS = ['contradiction', 'neutral', 'entailment']

# useful maps for when you must compare integers
SNLI_TO_MNLI_IDX = {SNLI_LABELS.index(name): MNLI_LABELS.index(name) for name in
MNLI_TO_SNLI_IDX = {v: k for k, v in SNLI_TO_MNLI_IDX.items()}

SCHEMA_VERSION = "1.1"   # single source of truth

# Set reproducible seeds
def set_all_seeds(seed=42):
    """Set all random seeds for reproducibility"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    print(f"✅ All seeds set to {seed}")

def get_git_hash():
    """Get current git hash for reproducibility"""
    try:
        git_hash = subprocess.check_output(['git', 'rev-parse', 'HEAD'],
                                           stderr=subprocess.DEVNULL).decode('asci
        return git_hash[:8]  # Short hash
    except Exception:
        return "no-git"

def save_run_config(dirs, **kwargs):
    """Save run configuration for reproducibility"""
    config = {
        "schema_version": SCHEMA_VERSION,
        "timestamp": datetime.datetime.now().isoformat(),
        "git_hash": get_git_hash(),
        "model_name": "roberta-large-mnli",
        "random_seed": 42,
        "num_samples": kwargs.get('num_samples', 50),
        "chunk_size": kwargs.get('chunk_size', 64),
        "lime_num_samples": kwargs.get('lime_num_samples', 1000),
        "lime_num_features": 10,
        "k_tokens": 3,
        "snli_labels": SNLI_LABELS,
        "mnli_labels": MNLI_LABELS,
        "snli_to_mnli_idx": SNLI_TO_MNLI_IDX,
    }

    config_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "run_config.json")
    write_json(config_path, config)
    print(f"✅ Run config saved to: {config_path}")
    return config

def _to_py(o):
    if isinstance(o, dict):
        return { _to_py(k): _to_py(v) for k, v in o.items() }
    if isinstance(o, (list, tuple)):
        return [ _to_py(v) for v in o ]
    if isinstance(o, (np.integer,)):
        return int(o)
```

```
    if isinstance(o, (np.floating,)):
        return float(o)
    if isinstance(o, np.ndarray):
        return o.tolist()
    return o  # fall back

def ensure_dir(path: str):
    os.makedirs(path, exist_ok=True)

def file_nonempty(path: str) -> bool:
    return os.path.isfile(path) and os.path.getsize(path) > 0

def read_json(path: str):
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def write_json(path: str, data):
    data = with_schema(data)
    ensure_dir(os.path.dirname(path))
    with open(path, "w", encoding="utf-8") as f:
        json.dump(_to_py(data), f, indent=2, ensure_ascii=False)

def with_schema(d):
    # attach schema unless the object already has it
    if isinstance(d, dict) and "schema_version" not in d:
        d = {"schema_version": SCHEMA_VERSION, **d}
    return d
```

Set Up Data Paths & Directories

In [3]:

```
def setup_directories():
    """Setup all required directories"""
    CWD = os.getcwd()
    DATA_DIR = os.path.join(CWD, "data")

    directories = {
        'DATA_DIR': DATA_DIR,
        'CACHE_DIR': os.path.join(DATA_DIR, "hf_cache"),
        'SNLI_LOCAL_DIR': os.path.join(DATA_DIR, "snli"),
        'SAMPLE_SNLI_DIR': os.path.join(DATA_DIR, "sampled_snli_data"),
        'PROCESSED_SNLI_DIR': os.path.join(DATA_DIR, "processed_snli_data"),
        'LIME_OUTPUT_DIR': os.path.join(DATA_DIR, "lime_outputs"),
    }

    # Create all directories
    for dir_path in directories.values():
        os.makedirs(dir_path, exist_ok=True)

    # Create plots subdirectory
    os.makedirs(os.path.join(directories['LIME_OUTPUT_DIR'], "plots"), exist_ok=

    return directories
```

Load Datasets and preview (with Local Cache)

Load SNLI dataset from disk if available, otherwise download and cache them locally.

In [4]:
```python
def load_snli_dataset_fixed(dirs):
    """Load SNLI dataset with proper error handling"""
    print("📥 Loading SNLI dataset...")

    dataset_path = dirs['SNLI_LOCAL_DIR']
    dataset_ready = os.path.exists(os.path.join(dataset_path, "dataset_dict.json

    if dataset_ready:
        print("Loading from local cache...")
        snli_data = load_from_disk(dataset_path)
    else:
        print("Downloading SNLI dataset...")
        snli_data = load_dataset("snli", cache_dir=dirs['CACHE_DIR'])
        snli_data.save_to_disk(dataset_path)

    # Validate dataset
    if 'train' not in snli_data:
        raise ValueError("SNLI dataset does not contain 'train' split.")

    sample = snli_data['train'][0]
    required_fields = ['premise', 'hypothesis', 'label']
    for field in required_fields:
        if field not in sample:
            raise ValueError(f"SNLI dataset missing field: {field}")

    print(f"✅ SNLI dataset loaded: {len(snli_data['train'])} training examples'
    print("Sample:", sample)

    return snli_data
```

Sample 300 Examples from Each Dataset and Save to disk

Randomly sample 300 valid examples from SNLI and CommonsenseQA for fast experimentation.

In [5]:
```python
def sample_snli_dataset_fixed(dataset, dirs, num_samples=300):
    """Ensure we have at least `num_samples` unique, valid items; extend if need
    sample_json_path = os.path.join(dirs['SAMPLE_SNLI_DIR'], "snli_sample.json")

    # start from existing
    existing = []
    if file_nonempty(sample_json_path):
        print(f"⏩ Found existing SNLI sample at {sample_json_path}; loading…")
        raw = read_json(sample_json_path)
        existing = raw.get("data", raw) if isinstance(raw, dict) else raw

    have = { (e["premise"], e["hypothesis"]) for e in existing }
    target = int(num_samples)

    if len(existing) >= target:
        print(f"✅ Already have {len(existing)} samples (>= {target}); reusing."
        return existing

    print(f"🎯 Need {target} samples; currently {len(existing)}. Extending…")
    # collect additional, unique, valid examples
    valid_indices = [i for i, ex in enumerate(dataset['train']) if ex['label'] !
    random.seed(42)
```

```
    random.shuffle(valid_indices)

    next_id = max([e.get("id", -1) for e in existing] + [-1]) + 1
    for idx in valid_indices:
        if len(existing) >= target:
            break
        ex = dataset['train'][idx]
        key = (ex['premise'], ex['hypothesis'])
        if key in have:
            continue
        existing.append({
            "id": next_id,
            "premise": ex["premise"],
            "hypothesis": ex["hypothesis"],
            "label": int(ex["label"]),
            "label_name": SNLI_LABELS[int(ex["label"])]
        })
        have.add(key); next_id += 1

    write_json(sample_json_path, with_schema({"data": existing}))
    print(f"✅ Saved {len(existing)} samples to: {sample_json_path}")
    return existing
```

Preprocess Sampled Data for Attribution

Convert SNLI and CSQA samples into model-ready format and save for later use in LIME/SHAP or other explainers.

In [6]:

```
def preprocess_snli_for_roberta(snli_sample_data, dirs):
    """
    Preprocess SNLI data specifically for RoBERTa-MNLI format
    Preprocess once; if processed file exists, load and return.
    """
    processed_path = os.path.join(dirs['PROCESSED_SNLI_DIR'], "processed_snli.js
    if file_nonempty(processed_path):
        print(f"⏩ Found existing processed SNLI at {processed_path}; loading…"
        return read_json(processed_path)

    print("🔄 Preprocessing SNLI data for RoBERTa-MNLI...")

    processed_snli = []
    for ex in snli_sample_data:
        # RoBERTa format: premise</s></s>hypothesis
        # But tokenizer handles this automatically, so we just use: premise<sep>
        text = f"{ex['premise']}</s>{ex['hypothesis']}"

        processed_entry = {
            "id": ex["id"],
            "input_text": text,
            "premise": ex["premise"],
            "hypothesis": ex["hypothesis"],
            "label": ex["label"],
            "label_name": ex["label_name"],
            "dataset": "snli"
        }
        processed_snli.append(processed_entry)

    # Save processed data
```

```
    write_json(processed_path, processed_snli)

    print(f"✅ Saved {len(processed_snli)} processed examples to: {processed_pa

    return processed_snli
```

PART 2: ROBERTA-MNLI MODEL SETUP

In [7]:
```
class RoBERTaMNLIClassifier:
    """Proper RoBERTa-MNLI classifier for LIME explanations"""

    def __init__(self, model_name="roberta-large-mnli", use_fp16=True):
        print(f"🤖 Loading {model_name} model...")

        self.model_name = model_name
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu
        print(f"Using device: {self.device}")

        # Load model and tokenizer
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForSequenceClassification.from_pretrained(model_na
        if use_fp16 and self.device.type == "cuda":
            self.model.half()
        self.model.to(self.device)
        self.model.eval()

        # Label mapping for MNLI (RoBERTa uses different order than SNLI)
        self.label_mapping = {i: name for i, name in enumerate(MNLI_LABELS)}

        print("✅ Model loaded successfully!")
        self._test_model()

    def _test_model(self):
        """Test model with a simple example"""
        print("🧪 Testing model...")

        test_premise = "The cat is sleeping on the couch."
        test_hypothesis = "The cat is awake."
        test_text = f"{test_premise}</s>{test_hypothesis}"

        probs = self.predict_proba([test_text])[0]
        predicted_label = int(np.argmax(probs))
        confidence = float(np.max(probs))

        print(f"Test input: '{test_premise}' vs '{test_hypothesis}'")
        print(f"Probabilities: {probs}")
        print(f"Predicted: {self.label_mapping[predicted_label]} (confidence: {c

        # Should predict contradiction with high confidence
        if predicted_label == 0 and confidence > 0.7:
            print("✅ Model test passed!")
        else:
            print("⚠️ Model test results seem unusual, but proceeding...")

    def predict_proba(self, texts):
        """Predict probabilities for LIME (batch processing)"""
        if isinstance(texts, str):
            texts = [texts]
```

```
        else:
            # normalize numpy/object arrays to a flat list of strings
            texts = [str(x) for x in np.array(texts, dtype=object).ravel().tolis

        premises, hyps = [], []
        for t in texts:
            if "</s>" in t:
                p, h = t.split("</s>", 1)
            else:
                # Fallback if LIME masks out the delimiter
                p, h = t, ""
            premises.append(p)
            hyps.append(h)

        inputs = self.tokenizer(
            premises,
            hyps,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=512
        )
        inputs = {k: v.to(self.device) for k, v in inputs.items()}

        with torch.no_grad():
            outputs = self.model(**inputs)
            probs = torch.softmax(outputs.logits, dim=1).detach().cpu().numpy()
        # Ensure 2D shape
        if probs.ndim == 1:
            probs = probs.reshape(1, -1)

        return probs

    def predict_single(self, text):
        """Get single prediction with details"""
        probs = self.predict_proba([text])[0]
        predicted_label = int(np.argmax(probs))

        return {
            'probabilities': probs,
            'predicted_label': predicted_label,
            'predicted_class': self.label_mapping[predicted_label],
            'confidence': float(np.max(probs)),
            'all_probs': {self.label_mapping[i]: probs[i] for i in range(len(pro
        }
```

PART 3: LIME EXPLANATIONS WITH PROPER EVALUATION

In [8]:
```
def generate_lime_explanations(classifier, processed_snli, dirs, num_examples=50
                               lime_num_samples=1000, chunk_size=64,
                               force_rebuild=False, save_every=10):
    """
    Generate LIME explanations with proper RoBERTa integration.
    Resumable: keeps existing items, appends new ones until num_examples.
    """
    lime_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "lime_explanations_roberta
```

```
    # Load any existing data (resumable)
    existing = []
    if file_nonempty(lime_path) and not force_rebuild:
        try:
            raw = read_json(lime_path)
            existing = raw.get("data", raw) if isinstance(raw, dict) else raw
            print(f"⏩ Found existing LIME explanations at {lime_path} with {len
        except Exception as e:
            print(f"⚠ Failed to read existing LIME file ({e}). Will regenerate.

    processed_ids = {int(r["id"]) for r in existing if "id" in r}
    target_total = int(num_examples)
    to_go = max(0, target_total - len(existing))
    if to_go == 0:
        print("⏩ Already have requested number of LIME items; returning cached
        return existing[:target_total]

    print(f"🔍 Generating LIME explanations for {to_go} new examples (target={ta
    print(f"   LIME samples: {lime_num_samples}, Chunk size: {chunk_size}")

    # Inner batched predictor with adaptive chunking + OOM recovery
    def _lime_predict(batch, chunk_size=chunk_size):
        if isinstance(batch, str):
            texts = [batch]
        else:
            texts = [str(x) for x in np.array(batch, dtype=object).ravel().tolis

        rows = []
        i = 0
        while i < len(texts):
            sub = texts[i:i+chunk_size]
            try:
                probs = classifier.predict_proba(sub)
            except RuntimeError as e:
                if "CUDA out of memory" in str(e) and chunk_size > 8:
                    torch.cuda.empty_cache()
                    chunk_size = max(8, chunk_size // 2)
                    print(f"⚠ OOM — reducing chunk_size to {chunk_size} and ret
                    continue
                raise
            probs = np.asarray(probs)
            if probs.ndim == 1: probs = probs.reshape(1, -1)
            # defensive per-item fallback
            if probs.shape[0] != len(sub):
                fixed = []
                for t in sub:
                    p = classifier.predict_proba([t])
                    p = np.asarray(p)
                    if p.ndim == 1: p = p.reshape(1, -1)
                    fixed.append(p[0])
                probs = np.vstack(fixed)
            rows.append(probs)
            i += len(sub)

        out = np.vstack(rows)
        assert out.shape[0] == len(texts), f"probs rows {out.shape[0]} != texts
        return out

    explainer = LimeTextExplainer(class_names=MNLI_LABELS, verbose=False, random
    new_items = []
```

```
    seen = 0

    for ex in tqdm(processed_snli, desc="LIME Explanations"):
        if len(existing) + len(new_items) >= target_total:
            break
        if int(ex["id"]) in processed_ids:
            continue

        try:
            text = ex["input_text"]
            t0 = time.perf_counter()
            prediction = classifier.predict_single(text)
            explanation = explainer.explain_instance(
                text,
                _lime_predict,
                num_features=10,
                num_samples=int(lime_num_samples),
            )
            latency = time.perf_counter() - t0
            item = {
                "id": int(ex["id"]),
                "premise": ex["premise"],
                "hypothesis": ex["hypothesis"],
                "input_text": text,
                "true_label": int(ex["label"]),
                "true_label_name": ex["label_name"],
                "predicted_label": int(prediction["predicted_label"]),
                "predicted_class": prediction["predicted_class"],
                "confidence": float(prediction["confidence"]),
                "all_probabilities": {k: float(v) for k, v in prediction["all_pr
                "lime_attributions": [(w, float(s)) for (w, s) in explanation.as
                "lime_score": float(explanation.score),
                "lime_runtime_sec": float(latency),
            }
            new_items.append(item)
            seen += 1

            # checkpoint
            if seen % save_every == 0:
                merged = {"schema_version": SCHEMA_VERSION, "data": existing + n
                write_json(lime_path, merged)
                print(f"💾 Checkpoint: saved {len(existing)+len(new_items)}/{tar

        except Exception as e:
            print(f"❌ Error on id={ex.get('id')}: {e}")

    merged = {"schema_version": SCHEMA_VERSION, "data": existing + new_items}
    write_json(lime_path, merged)
    print(f"✅ Generated {len(new_items)} new LIME explanations (total={len(exis
    print(f"✅ Saved to: {lime_path}")
    return (existing + new_items)[:target_total]
```

In [9]:

```
def filter_stopwords_from_lime(lime_results, dirs):
    """Remove stopwords from LIME attributions for cleaner analysis"""
    if not lime_results:
        print("⏩ No LIME results; skipping stopword filtering.")
        return []
    print("🧹 Filtering stopwords from LIME attributions...")

    # Setup stopwords
```

```
    stopwords_set = set(stopwords.words('english'))
    additional_stopwords = {"</s>", "[SEP]", "[CLS]", "[PAD]", "[UNK]", "[MASK]"
    stopwords_set.update(additional_stopwords)

    filtered_results = []
    for result in lime_results:
        # Filter attributions
        filtered_attributions = [
            (token, score) for token, score in result["lime_attributions"]
            if token.lower() not in stopwords_set and len(token.strip()) > 1
        ]

        # Create filtered result
        filtered_result = result.copy()
        filtered_result["lime_attributions_filtered"] = filtered_attributions
        filtered_result["original_attribution_count"] = len(result["lime_attribu
        filtered_result["filtered_attribution_count"] = len(filtered_attribution

        filtered_results.append(filtered_result)

    # Save filtered results with schema
    filtered_data_with_schema = {
        "schema_version": SCHEMA_VERSION,
        "data": filtered_results
    }
    filtered_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "lime_explanations_fil
    write_json(filtered_path, filtered_data_with_schema)

    print(f"✅ Filtered results saved to: {filtered_path}")
    print(f"Average attribution reduction: {np.mean([r['original_attribution_cou

    return filtered_results
```

PART 4: EVALUATION METRICS

In [10]:

```
def _split_pair(text):
    if "</s>" in text:
        return text.split("</s>", 1)
    return text, ""

def compute_faithfulness(classifier, original_text, top_tokens):
    try:
        p, h = _split_pair(original_text)
        words = h.split()
        perturbed_h = " ".join([w for w in words if w not in top_tokens])
        if not perturbed_h.strip(): return 0.0
        orig = classifier.predict_proba([original_text])[0].max()
        pert = classifier.predict_proba([f"{p}</s>{perturbed_h}"])[0].max()
        return abs(orig - pert)
    except Exception:
        return 0.0

def compute_sufficiency(classifier, original_text, top_tokens):
    try:
        p, h = _split_pair(original_text)
        words = h.split()
        kept_h = " ".join([w for w in words if w in top_tokens])
        if not kept_h.strip(): return 0.0
```

```
        return classifier.predict_proba([f"{p}</s>{kept_h}"])[0].max()
    except Exception:
        return 0.0

def compute_comprehensiveness(classifier, original_text, top_tokens):
    try:
        p, h = _split_pair(original_text)
        words = h.split()
        remain_h = " ".join([w for w in words if w not in top_tokens])
        if not remain_h.strip(): return 0.0
        orig = classifier.predict_proba([original_text])[0].max()
        rem  = classifier.predict_proba([f"{p}</s>{remain_h}"])[0].max()
        return orig - rem
    except Exception:
        return 0.0
```

In [11]:

```
def compute_evaluation_metrics(classifier, lime_results, k=3, dirs=None, save_ev
    """Compute faithfulness, sufficiency, and comprehensiveness"""
    print(f"📊 Computing evaluation metrics (k={k})...")
    out_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "metrics_partial.json") if

    done_ids = set()
    partial = []
    if out_path and file_nonempty(out_path):
        try:
            raw = read_json(out_path)
            partial = raw.get("data", raw) if isinstance(raw, dict) else raw
            done_ids = {int(r["id"]) for r in partial if "id" in r}
            print(f"⏩ Resuming metrics from {len(done_ids)} items.")
        except Exception:
            pass

    results = list(partial)
    since_last = 0

    for result in tqdm(lime_results, desc="Computing metrics"):
        try:
            text = result["input_text"]
            attributions = result.get("lime_attributions_filtered", result["lime

            if len(attributions) < k:
                continue

            rid = int(result["id"])
            if rid in done_ids:
                continue

            # Get top-k tokens
            sorted_attrs = sorted(attributions, key=lambda x: abs(x[1]), reverse
            top_tokens = [token for token, _ in sorted_attrs[:k]]

            # Compute metrics
            faithfulness = compute_faithfulness(classifier, text, top_tokens)
            sufficiency = compute_sufficiency(classifier, text, top_tokens)
            comprehensiveness = compute_comprehensiveness(classifier, text, top_

            metric_result = {
                "id": rid,
                "faithfulness": faithfulness,
                "sufficiency": sufficiency,
```

```
                "comprehensiveness": comprehensiveness,
                "top_tokens": top_tokens,
                "k": k
            }

            results.append(metric_result)
            since_last += 1

            if out_path and since_last >= save_every:
                write_json(out_path, with_schema({"data": results}))
                since_last = 0

        except Exception as e:
            print(f"❌ Error computing metrics for example {result['id']}: {e}")
            continue

    if out_path:
        write_json(out_path, with_schema({"data": results}))

    return results
```

In [12]:

```
def perform_sanity_checks(classifier, lime_results, dirs):
    """Perform sanity checks on explanations"""
    print("🔍 Performing sanity checks...")

    # Separate correct and incorrect predictions
    to_mnli = {'contradiction':0,'neutral':1,'entailment':2}
    correct_examples = [r for r in lime_results if to_mnli[r['true_label_name']]
    incorrect_examples = [r for r in lime_results if to_mnli[r['true_label_name'

    print(f"Found {len(correct_examples)} correct, {len(incorrect_examples)} inc

    sanity_results = {
        "correct_examples": [],
        "incorrect_examples": [],
        "shuffle_test_results": []
    }

    # Check top 5 correct examples
    for i, example in enumerate(correct_examples[:5]):
        attributions = example.get("lime_attributions_filtered", example["lime_a
        top_5_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=Tru

        sanity_results["correct_examples"].append({
            "id": example["id"],
            "predicted_class": example["predicted_class"],
            "confidence": example["confidence"],
            "top_tokens": [(token, float(score)) for token, score in top_5_token
            "premise": example["premise"][:100] + "..." if len(example["premise"
            "hypothesis": example["hypothesis"][:100] + "..." if len(example["hy
        })

    # Check top 5 incorrect examples
    for i, example in enumerate(incorrect_examples[:5]):
        attributions = example.get("lime_attributions_filtered", example["lime_a
        top_5_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=Tru

        sanity_results["incorrect_examples"].append({
            "id": example["id"],
            "true_class": example["true_label_name"],
```

```
            "predicted_class": example["predicted_class"],
            "confidence": example["confidence"],
            "top_tokens": [(token, float(score)) for token, score in top_5_token
            "premise": example["premise"][:100] + "..." if len(example["premise"
            "hypothesis": example["hypothesis"][:100] + "..." if len(example["hy
        })

    # Shuffle test on 3 examples
    print("🔀 Running shuffle sanity test...")
    for i, example in enumerate(lime_results[:3]):
        original_text = example["input_text"]

        # Shuffle words
        words = original_text.split()
        random.shuffle(words)
        shuffled_text = " ".join(words)

        # Compute faithfulness for both
        attributions = example.get("lime_attributions_filtered", example["lime_a
        if len(attributions) >= 3:
            top_tokens = [token for token, _ in sorted(attributions, key=lambda

            original_faithfulness = compute_faithfulness(classifier, original_te
            shuffled_faithfulness = compute_faithfulness(classifier, shuffled_te

            sanity_results["shuffle_test_results"].append({
                "id": example["id"],
                "original_faithfulness": float(original_faithfulness),
                "shuffled_faithfulness": float(shuffled_faithfulness),
                "faithfulness_drop": float(original_faithfulness - shuffled_fait
            })

    # Save sanity check results
    sanity_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "sanity_check_results.js
    write_json(sanity_path, sanity_results)
    print(f"✅ Sanity check results saved to: {sanity_path}")

    return sanity_results
```

In [13]:

```
def error_analysis(lime_results, dirs):
    """Analyze errors and create confusion matrix"""
    print("🔍 Performing error analysis...")

    # Get 10 most confident wrong predictions
    wrong_predictions = [r for r in lime_results if r['true_label'] != r['predic
    most_confident_wrong = sorted(wrong_predictions, key=lambda x: x['confidence

    error_analysis_results = []
    for example in most_confident_wrong:
        attributions = example.get("lime_attributions_filtered", example["lime_a
        top_10_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=Tr

        error_analysis_results.append({
            "id": example["id"],
            "premise": example["premise"],
            "hypothesis": example["hypothesis"],
            "true_label": example["true_label_name"],
            "predicted_label": example["predicted_class"],
            "confidence": example["confidence"],
            "top_10_attributions": [(token, float(score)) for token, score in to
```

```
        })

    # Create confusion matrix
    y_true = [SNLI_TO_MNLI_IDX[int(r['true_label'])] for r in lime_results]   #
    y_pred = [int(r['predicted_label']) for r in lime_results]

    cm = confusion_matrix(y_true, y_pred, labels=[0, 1, 2])  # MNLI indices: 0=c
    label_names = ['entailment', 'neutral', 'contradiction']

    # Save error analysis
    error_data = {
        "schema_version": SCHEMA_VERSION,
        "most_confident_wrong_predictions": error_analysis_results,
        "confusion_matrix": cm.tolist(),
        "label_names": label_names,
        "summary": {
            "total_examples": len(lime_results),
            "wrong_predictions": len(wrong_predictions),
            "error_rate": len(wrong_predictions) / len(lime_results)
        }
    }

    error_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "error_analysis.json")
    write_json(error_path, error_data)
    print(f"✅ Error analysis saved to: {error_path}")

    return error_data
```

In [14]:

```
def lime_stability_sweep(classifier, processed_snli, dirs):
    """Mini parameter sweep for LIME stability"""
    print("🔬 Running LIME stability mini-sweep...")

    stability_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "lime_stability_sweep
    # Parameter grid
    param_grid = [
        {"num_samples": 500, "kernel_width": None},
        {"num_samples": 1000, "kernel_width": None},
        # {"num_samples": 500, "kernel_width": 25},
        # {"num_samples": 1000, "kernel_width": 25}
    ]

    # Test on 10 examples
    test_examples = processed_snli[:10]  # keep small so it's fast/visible
    total_tasks = len(param_grid) * len(test_examples)

    # reuse the robust LIME callback you wrote earlier
    def _lime_predict(batch):
        if isinstance(batch, str):
            texts = [batch]
        else:
            texts = [str(x) for x in np.array(batch, dtype=object).ravel().tolis
        return classifier.predict_proba(texts)

    stability_results = []
    with tqdm(total=total_tasks, desc="Stability sweep", unit="ex", dynamic_ncol
        for pi, params in enumerate(param_grid, 1):
            kw = params["kernel_width"]
            kw_label = "default" if kw is None else kw

            # Build explainer (omit kernel_width when None)
```

```
            init_kwargs = {
                "class_names": list(classifier.label_mapping.values()),
                "verbose": False,
                "random_state": 42,
            }
            if kw is not None:
                init_kwargs["kernel_width"] = float(kw)

            explainer = LimeTextExplainer(**init_kwargs)

            metrics = []
            for ei, ex in enumerate(test_examples, 1):
                try:
                    text = ex["input_text"]
                    explanation = explainer.explain_instance(
                        text,
                        _lime_predict,
                        num_features=10,
                        num_samples=int(params["num_samples"]),
                    )
                    attributions = explanation.as_list()
                    if len(attributions) >= 3:
                        top_tokens = [t for t, _ in sorted(attributions, key=lam
                        f = compute_faithfulness(classifier, text, top_tokens)
                        s = compute_sufficiency(classifier, text, top_tokens)
                        metrics.append({"faithfulness": float(f), "sufficiency":

                except Exception as e:
                    tqdm.write(f"Error (ns={params['num_samples']}, kw={kw_label

                # update the global bar + postfix info
                pbar.set_postfix_str(f"ns={params['num_samples']}, kw={kw_label}
                pbar.update(1)

            if metrics:
                stability_results.append({
                    "params": params,
                    "avg_faithfulness": float(np.mean([m["faithfulness"] for m i
                    "avg_sufficiency": float(np.mean([m["sufficiency"] for m in
                    "num_examples": len(metrics),
                })
                write_json(stability_path, with_schema({"results": stability_res

    # Save stability results
    stability_data = {
        "schema_version": SCHEMA_VERSION,
        "results": stability_results
    }
    write_json(stability_path, stability_data)
    print(f"✅ Stability sweep results saved to: {stability_path}")

    return stability_results
```

In [24]:

```
def create_comprehensive_plots(lime_results, metrics_results, dirs):
    """Create comprehensive visualization plots"""
    print("📊 Creating comprehensive visualization plots...")

    plots_dir = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots")
    ensure_dir(plots_dir)
```

```
# Set style
plt.style.use('default')
sns.set_palette("husl")

# 1. Model Performance Analysis
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# Confidence distribution
confidences = [r['confidence'] for r in lime_results]
axes[0, 0].hist(confidences, bins=20, alpha=0.7, color='skyblue', edgecolor=
axes[0, 0].set_title('Model Confidence Distribution')
axes[0, 0].set_xlabel('Confidence Score')
axes[0, 0].set_ylabel('Frequency')
axes[0, 0].axvline(np.mean(confidences), color='red', linestyle='--',
                   label=f'Mean: {np.mean(confidences):.3f}')
axes[0, 0].legend()

# Label distribution
true_labels = [r['true_label_name'] for r in lime_results]
pred_labels = [r['predicted_class'] for r in lime_results]

label_counts = Counter(true_labels)
axes[0, 1].bar(label_counts.keys(), label_counts.values(), alpha=0.7, color=
axes[0, 1].set_title('True Label Distribution')
axes[0, 1].set_xlabel('Labels')
axes[0, 1].set_ylabel('Count')
axes[0, 1].tick_params(axis='x', rotation=45)

# Accuracy by confidence bins
df_results = pd.DataFrame(lime_results)
df_results['correct'] = df_results['true_label_name'] == df_results['predict
df_results['conf_bin'] = pd.cut(df_results['confidence'], bins=5,
                                labels=['Very Low', 'Low', 'Medium', 'High',
conf_acc = df_results.groupby('conf_bin')['correct'].mean()

axes[1, 0].bar(range(len(conf_acc)), conf_acc.values, alpha=0.7, color='oran
axes[1, 0].set_title('Accuracy by Confidence Level')
axes[1, 0].set_xlabel('Confidence Bins')
axes[1, 0].set_ylabel('Accuracy')
axes[1, 0].set_xticks(range(len(conf_acc)))
axes[1, 0].set_xticklabels(conf_acc.index, rotation=45)

# Confusion matrix
true_labels_numeric = [r['true_label_name'] for r in lime_results]
pred_labels_numeric = [r['predicted_class'] for r in lime_results]
label_names = ['entailment', 'neutral', 'contradiction']
cm = confusion_matrix(true_labels_numeric, pred_labels_numeric, labels=label

im = axes[1, 1].imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
axes[1, 1].set_title('Confusion Matrix')
tick_marks = np.arange(len(label_names))
axes[1, 1].set_xticks(tick_marks)
axes[1, 1].set_yticks(tick_marks)
axes[1, 1].set_xticklabels(label_names, rotation=45)
axes[1, 1].set_yticklabels(label_names)

# Add text annotations
thresh = cm.max() / 2.
for i, j in np.ndindex(cm.shape):
    axes[1, 1].text(j, i, format(cm[i, j], 'd'),
```

```
                    ha="center", va="center",
                    color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.savefig(os.path.join(plots_dir, "model_performance_analysis.png"), dpi=3
    plt.close()

    # 2. LIME Attribution Analysis
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # Attribution score distribution
    all_scores = []
    for result in lime_results:
        attributions = result.get('lime_attributions_filtered', result['lime_att
        scores = [abs(score) for _, score in attributions]
        all_scores.extend(scores)

    axes[0, 0].hist(all_scores, bins=30, alpha=0.7, color='purple', edgecolor='b
    axes[0, 0].set_title('LIME Attribution Score Distribution')
    axes[0, 0].set_xlabel('Absolute Attribution Score')
    axes[0, 0].set_ylabel('Frequency')
    axes[0, 0].axvline(np.mean(all_scores), color='red', linestyle='--',
                    label=f'Mean: {np.mean(all_scores):.3f}')
    axes[0, 0].legend()

    # LIME score distribution
    lime_scores = [r['lime_score'] for r in lime_results]
    axes[0, 1].hist(lime_scores, bins=20, alpha=0.7, color='coral', edgecolor='b
    axes[0, 1].set_title('LIME Score Distribution')
    axes[0, 1].set_xlabel('LIME Score')
    axes[0, 1].set_ylabel('Frequency')
    axes[0, 1].axvline(np.mean(lime_scores), color='red', linestyle='--',
                    label=f'Mean: {np.mean(lime_scores):.3f}')
    axes[0, 1].legend()

    # Confidence vs Faithfulness scatter (if metrics available)
    if metrics_results:
        # Create mapping from id to metrics
        metrics_dict = {m['id']: m for m in metrics_results}

        plot_data = []
        for result in lime_results:
            if result['id'] in metrics_dict:
                plot_data.append({
                    'confidence': result['confidence'],
                    'faithfulness': metrics_dict[result['id']]['faithfulness']
                })

        if plot_data:
            conf_vals = [d['confidence'] for d in plot_data]
            faith_vals = [d['faithfulness'] for d in plot_data]

            axes[1, 0].scatter(conf_vals, faith_vals, alpha=0.6, color='green')
            axes[1, 0].set_title('Confidence vs Faithfulness')
            axes[1, 0].set_xlabel('Model Confidence')
            axes[1, 0].set_ylabel('Faithfulness Score')

            # Add correlation
            correlation = np.corrcoef(conf_vals, faith_vals)[0, 1]
            axes[1, 0].text(0.05, 0.95, f'Correlation: {correlation:.3f}',
```

```
                        transform=axes[1, 0].transAxes, fontsize=12,
                        verticalalignment='top', bbox=dict(boxstyle='round',

# Top contributing words
word_scores = defaultdict(list)
for result in lime_results:
    attributions = result.get('lime_attributions_filtered', result['lime_att
    for word, score in attributions:
        word_scores[word.lower()].append(abs(score))

# Get top 15 words by average absolute score
avg_word_scores = {word: np.mean(scores) for word, scores in word_scores.ite
top_words = sorted(avg_word_scores.items(), key=lambda x: x[1], reverse=True

words, scores = zip(*top_words)
axes[1, 1].barh(range(len(words)), scores, alpha=0.7, color='teal')
axes[1, 1].set_yticks(range(len(words)))
axes[1, 1].set_yticklabels(words)
axes[1, 1].set_title('Top 15 Contributing Words')
axes[1, 1].set_xlabel('Average Absolute Attribution Score')

plt.tight_layout()
plt.savefig(os.path.join(plots_dir, "lime_attribution_analysis.png"), dpi=30
plt.close()

# 3. Metrics Analysis (if available)
if metrics_results:
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    metrics_df = pd.DataFrame(metrics_results)

    # Metrics comparison
    metric_means = [metrics_df['faithfulness'].mean(),
                    metrics_df['sufficiency'].mean(),
                    metrics_df['comprehensiveness'].mean()]
    metric_names = ['Faithfulness', 'Sufficiency', 'Comprehensiveness']

    bars = axes[0].bar(metric_names, metric_means, alpha=0.7,
                       color=['red', 'blue', 'green'])
    axes[0].set_title('Average Attribution Metrics')
    axes[0].set_ylabel('Score')
    axes[0].tick_params(axis='x', rotation=45)

    # Add value labels on bars
    for bar, value in zip(bars, metric_means):
        axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.0
                     f'{value:.3f}', ha='center', va='bottom')

    # Metrics correlation
    correlation_matrix = metrics_df[['faithfulness', 'sufficiency', 'compreh

    im = axes[1].imshow(correlation_matrix, cmap='RdYlBu', vmin=-1, vmax=1)
    axes[1].set_title('Metrics Correlation Matrix')
    axes[1].set_xticks(range(len(correlation_matrix.columns)))
    axes[1].set_yticks(range(len(correlation_matrix.columns)))
    axes[1].set_xticklabels(correlation_matrix.columns, rotation=45)
    axes[1].set_yticklabels(correlation_matrix.columns)

    # Add correlation values
    for i in range(len(correlation_matrix.columns)):
```

```
            for j in range(len(correlation_matrix.columns)):
                axes[1].text(j, i, f'{correlation_matrix.iloc[i, j]:.2f}',
                             ha='center', va='center', color='white', fontweight=

        plt.colorbar(im, ax=axes[1])
        plt.tight_layout()
        plt.savefig(os.path.join(plots_dir, "metrics_analysis.png"), dpi=300, bb
        plt.close()

    print(f"✅ All plots saved to: {plots_dir}")
```

In [16]:

```
def bootstrap_confidence_intervals(metrics_results, n_bootstrap=1000):
    """Compute bootstrap confidence intervals for metrics"""
    if not metrics_results:
        return {}

    print("🔄 Computing bootstrap confidence intervals...")

    metrics_df = pd.DataFrame(metrics_results)
    bootstrap_results = {}

    for metric in ['faithfulness', 'sufficiency', 'comprehensiveness']:
        if metric in metrics_df.columns:
            values = metrics_df[metric].values
            bootstrap_means = []

            for _ in range(n_bootstrap):
                # Resample with replacement
                resampled = np.random.choice(values, size=len(values), replace=T
                bootstrap_means.append(np.mean(resampled))

            # Calculate 95% CI
            ci_lower = np.percentile(bootstrap_means, 2.5)
            ci_upper = np.percentile(bootstrap_means, 97.5)

            bootstrap_results[metric] = {
                "mean": float(np.mean(values)),
                "ci_lower": float(ci_lower),
                "ci_upper": float(ci_upper),
                "std": float(np.std(bootstrap_means))
            }

    print("✅ Bootstrap confidence intervals computed")
    return bootstrap_results

def generate_report(lime_results, metrics_results, dirs, config):
    """Generate comprehensive markdown report"""
    print("📝 Generating SNLI LIME report...")

    # Calculate basic statistics
    total_examples = len(lime_results)
    correct_predictions = sum(1 for r in lime_results if r['true_label_name'] ==
    accuracy = correct_predictions / total_examples
    avg_confidence = np.mean([r['confidence'] for r in lime_results])

    # Metrics statistics
    metrics_stats = {}
    if metrics_results:
        metrics_df = pd.DataFrame(metrics_results)
        for metric in ['faithfulness', 'sufficiency', 'comprehensiveness']:
```

```
            if metric in metrics_df.columns:
                metrics_stats[metric] = {
                    'mean': metrics_df[metric].mean(),
                    'std': metrics_df[metric].std(),
                    'min': metrics_df[metric].min(),
                    'max': metrics_df[metric].max()
                }

    # Bootstrap CIs
    bootstrap_cis = bootstrap_confidence_intervals(metrics_results)

    # Select qualitative examples
    # Top 3 correct examples by confidence
    correct_examples = [r for r in lime_results if r['true_label_name'] == r['pr
    top_correct = sorted(correct_examples, key=lambda x: x['confidence'], revers

    # Top 2 incorrect examples by confidence
    incorrect_examples = [r for r in lime_results if r['true_label_name'] != r['
    top_incorrect = sorted(incorrect_examples, key=lambda x: x['confidence'], re

    # Generate report content
    report_content = f"""# SNLI LIME Explanation Analysis Report

## Executive Summary

**Dataset**: SNLI (Stanford Natural Language Inference)
**Model**: {config['model_name']}
**Analysis Date**: {config['timestamp'][:10]}
**Total Examples**: {total_examples}
**Git Hash**: {config['git_hash']}

### Key Results
- **Model Accuracy**: {accuracy:.3f} ({correct_predictions}/{total_examples})
- **Average Confidence**: {avg_confidence:.3f}
- **LIME Parameters**: {config['lime_num_samples']} samples, {config['lime_num_f
- **Random Seed**: {config['random_seed']}

## Attribution Metrics Results

"""

    if metrics_stats:
        report_content += "### Faithfulness, Sufficiency, and Comprehensiveness\
        report_content += "| Metric | Mean | Std | Min | Max | 95% CI |\n"
        report_content += "|--------|------|-----|-----|-----|--------|\n"

        for metric, stats in metrics_stats.items():
            ci_info = ""
            if metric in bootstrap_cis:
                ci = bootstrap_cis[metric]
                ci_info = f"[{ci['ci_lower']:.3f}, {ci['ci_upper']:.3f}]"

            report_content += f"| {metric.title()} | {stats['mean']:.3f} | {stat

    report_content += "\n## LIME Configuration\n\n"
    report_content += f"- **Number of Samples**: {config['lime_num_samples']}\n"
    report_content += f"- **Number of Features**: {config['lime_num_features']}\
    report_content += f"- **Top-k Tokens for Metrics**: {config['k_tokens']}\n"
    report_content += f"- **Chunk Size**: {config['chunk_size']}\n"
```

```
    report_content += "\n## Qualitative Examples\n\n"
    report_content += "### Top Correct Predictions\n\n"

    for i, example in enumerate(top_correct):
        attributions = example.get('lime_attributions_filtered', example['lime_a
        top_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=True)

        report_content += f"**Example {i+1}** (Confidence: {example['confidence'
        report_content += f"- **Premise**: {example['premise']}\n"
        report_content += f"- **Hypothesis**: {example['hypothesis']}\n"
        report_content += f"- **Predicted**: {example['predicted_class']}\n"
        report_content += f"- **Top Attributions**: "

        token_strs = [f"{token}({score:.3f})" for token, score in top_tokens]
        report_content += ", ".join(token_strs) + "\n\n"

    if top_incorrect:
        report_content += "### Top Incorrect Predictions\n\n"

        for i, example in enumerate(top_incorrect):
            attributions = example.get('lime_attributions_filtered', example['li
            top_tokens = sorted(attributions, key=lambda x: abs(x[1]), reverse=T

            report_content += f"**Example {i+1}** (Confidence: {example['confide
            report_content += f"- **Premise**: {example['premise']}\n"
            report_content += f"- **Hypothesis**: {example['hypothesis']}\n"
            report_content += f"- **True**: {example['true_label_name']}, **Pred
            report_content += f"- **Top Attributions**: "

            token_strs = [f"{token}({score:.3f})" for token, score in top_tokens
            report_content += ", ".join(token_strs) + "\n\n"

    report_content += "\n## Known Caveats\n\n"
    report_content += "- **Delimiter Handling**: Uses `</s>` separator between p
    report_content += "- **Stopword Filtering**: Removes common English stopword
    report_content += "- **Limited Sample Size**: Analysis based on 50 examples
    report_content += "- **Perturbation Sensitivity**: LIME results may vary wit

    report_content += "\n## Next Steps for GRACE Framework\n\n"
    report_content += "1. **Dynamic Retrieval-Guided Attribution (DRGA)**: Imple
    report_content += "2. **Explanation Hallucination Detection**: Build 4-type
    report_content += "3. **Fairness Auditor**: Add demographic bias detection\n
    report_content += "4. **Unified Trust Score**: Combine faithfulness + factua

    # Save report
    report_path = os.path.join(dirs['LIME_OUTPUT_DIR'], "SNLI_LIME_report.md")
    with open(report_path, "w", encoding="utf-8") as f:
        f.write(report_content)

    print(f"✅ Report saved to: {report_path}")
    return report_content

def analyze_results_enhanced(lime_results, metrics_results, dirs):
    """Enhanced results analysis with comprehensive statistics"""
    print("📈 Enhanced results analysis...")

    if not lime_results:
        print("⏩ No results; skipping analysis.")
        return {}
```

```
# Basic performance metrics
total = len(lime_results)
correct = sum(1 for r in lime_results if r.get("true_label") == r.get("predi
accuracy = float(correct / total) if total else 0.0
avg_confidence = float(np.mean([r.get("confidence", 0) for r in lime_results

# Metrics analysis
metrics_summary = {}
if metrics_results:
    metrics_df = pd.DataFrame(metrics_results)
    metrics_summary = {
        "avg_faithfulness": float(metrics_df['faithfulness'].mean()),
        "avg_sufficiency": float(metrics_df['sufficiency'].mean()),
        "avg_comprehensiveness": float(metrics_df['comprehensiveness'].mean(
        "std_faithfulness": float(metrics_df['faithfulness'].std()),
        "std_sufficiency": float(metrics_df['sufficiency'].std()),
        "std_comprehensiveness": float(metrics_df['comprehensiveness'].std()
        "num_examples": len(metrics_results)
    }

# Confidence distribution analysis
confidences = [r['confidence'] for r in lime_results]
confidence_analysis = {
    "high_confidence": sum(1 for c in confidences if c > 0.8),
    "medium_confidence": sum(1 for c in confidences if 0.5 < c <= 0.8),
    "low_confidence": sum(1 for c in confidences if c <= 0.5),
    "avg_confidence": float(np.mean(confidences)),
    "std_confidence": float(np.std(confidences))
}

# Build final results
final_results = {
    "schema_version": SCHEMA_VERSION,
    "evaluation_metrics": metrics_summary,
    "model_performance": {
        "accuracy": accuracy,
        "avg_confidence": avg_confidence,
        "correct_predictions": int(correct),
        "total_examples": int(total),
        "error_rate": float(1 - accuracy)
    },
    "confidence_analysis": confidence_analysis,
    "lime_explanations": lime_results,
    "individual_metrics": metrics_results if metrics_results else []
}

# Save results
results_path = os.path.join(dirs["LIME_OUTPUT_DIR"], "complete_evaluation_re
write_json(results_path, final_results)

print(f"\n🎯 FINAL EVALUATION RESULTS:")
print(f"   Model Accuracy: {accuracy:.3f}")
print(f"   Average Confidence: {avg_confidence:.3f}")

if metrics_summary:
    print(f"   Average Faithfulness: {metrics_summary['avg_faithfulness']:.3
    print(f"   Average Sufficiency: {metrics_summary['avg_sufficiency']:.3f}
    print(f"   Average Comprehensiveness: {metrics_summary['avg_comprehensiv

print(f"✅ Complete results saved to: {results_path}")
```

```
    return final_results

def plot_runtime_distribution(lime_results, dirs):
    print("📈 Plotting LIME runtime distribution...")
    times = [r.get("lime_runtime_sec", np.nan) for r in lime_results]
    times = [t for t in times if not np.isnan(t)]
    if not times: return
    plt.figure(figsize=(8,5))
    plt.hist(times, bins=20, edgecolor="black", alpha=0.7)
    plt.title("Per-example LIME runtime")
    plt.xlabel("Seconds"); plt.ylabel("Count")
    outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots", "lime_runtime_hist.png
    ensure_dir(os.path.dirname(outp))
    plt.tight_layout(); plt.savefig(outp, dpi=300); plt.close()

def plot_per_class_top_words(lime_results, dirs, by="predicted_class", top_n=15)
    print("📈 Plotting top words per class...")
    groups = {"entailment": [], "neutral": [], "contradiction": []}
    for r in lime_results:
        lbl = r.get(by)
        if lbl not in groups: continue
        atts = r.get("lime_attributions_filtered", r.get("lime_attributions", []
        groups[lbl].extend([ (w.lower(), abs(s)) for w,s in atts ])

    for lbl, pairs in groups.items():
        if not pairs: continue
        from collections import defaultdict
        agg = defaultdict(list)
        for w,s in pairs: agg[w].append(s)
        avg = sorted(((w, float(np.mean(v))) for w,v in agg.items()),
                     key=lambda x: x[1], reverse=True)[:top_n]
        words, scores = zip(*avg)
        plt.figure(figsize=(8,6))
        plt.barh(range(len(words)), scores)
        plt.yticks(range(len(words)), words)
        plt.gca().invert_yaxis()
        plt.title(f"Top {top_n} contributing words — {lbl}")
        plt.xlabel("Avg |attribution|")
        outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots", f"top_words_{lbl}.
        ensure_dir(os.path.dirname(outp))
        plt.tight_layout(); plt.savefig(outp, dpi=300); plt.close()

def plot_reliability_diagram(lime_results, dirs, bins=10):
    print("📈 Plotting reliability diagram...")
    # max prob & correctness
    preds = [r["all_probabilities"].get(r["predicted_class"], r["confidence"]) f
    correct = [int(r["true_label_name"] == r["predicted_class"]) for r in lime_r
    preds = np.array(preds, dtype=float)
    correct = np.array(correct, dtype=float)

    # bin by confidence
    edges = np.linspace(0, 1, bins+1)
    idx = np.digitize(preds, edges) - 1
    acc, conf, cnt = [], [], []
    for b in range(bins):
        mask = idx == b
        if mask.any():
            acc.append(float(correct[mask].mean()))
            conf.append(float(preds[mask].mean()))
            cnt.append(int(mask.sum()))
```

```
        else:
            acc.append(np.nan); conf.append((edges[b]+edges[b+1])/2); cnt.append

    plt.figure(figsize=(6,6))
    plt.plot([0,1],[0,1], '--', label='perfect')
    plt.plot(conf, acc, marker='o', label='model')
    plt.xlabel('Mean predicted confidence'); plt.ylabel('Empirical accuracy')
    plt.title('Reliability diagram'); plt.legend()
    outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "plots", "reliability.png")
    ensure_dir(os.path.dirname(outp))
    plt.tight_layout(); plt.savefig(outp, dpi=300); plt.close()
```

In [17]:

```
def analyze_results(filtered_results, metrics_results, dirs):
    """Save a summary, accepting metrics_results as dict OR list OR (dict, list)
    if not filtered_results:
        print("⏩ No results; skipping analysis.")
        return {}

    def safe_float(x):
        try:
            return float(x)
        except Exception:
            return np.nan

    # 1) Normalize metrics_results into summary numbers
    avg_faithfulness = np.nan
    avg_sufficiency = np.nan
    avg_comprehensiveness = np.nan
    n = None
    per_example = None

    if isinstance(metrics_results, dict):
        # expected keys already there
        avg_faithfulness = safe_float(metrics_results.get("avg_faithfulness"))
        avg_sufficiency = safe_float(metrics_results.get("avg_sufficiency"))
        avg_comprehensiveness = safe_float(metrics_results.get("avg_comprehensiv
        n = int(metrics_results.get("n", len(filtered_results)))
        per_example = metrics_results.get("per_example")

    elif isinstance(metrics_results, (list, tuple)):
        # tuple like (summary_dict, per_list)?
        if isinstance(metrics_results, tuple) and len(metrics_results) == 2 and
            summ, per = metrics_results
            avg_faithfulness = safe_float(summ.get("avg_faithfulness"))
            avg_sufficiency = safe_float(summ.get("avg_sufficiency"))
            avg_comprehensiveness = safe_float(summ.get("avg_comprehensiveness")
            n = int(summ.get("n", len(filtered_results)))
            per_example = per
        else:
            # treat as list of per-example dicts
            per_example = list(metrics_results)
            n = len(per_example)
            if n:
                avg_faithfulness = float(np.nanmean([safe_float(x.get("faithfuln
                avg_sufficiency = float(np.nanmean([safe_float(x.get("sufficienc
                avg_comprehensiveness = float(np.nanmean([safe_float(x.get("comp
            else:
                avg_faithfulness = avg_sufficiency = avg_comprehensiveness = flo
    else:
        # unknown type; fall back to computing only from filtered_results
```

```
        n = len(filtered_results)

    # 2) Model performance from filtered_results (always available)
    total = len(filtered_results)
    correct = sum(1 for r in filtered_results if r.get("true_label_name") == r.g
    avg_confidence = float(np.mean([safe_float(r.get("confidence")) for r in fil
    accuracy = float(correct / total) if total else 0.0

    # 3) Build final summary (cast to native Python types)
    final_results = {
        "evaluation": {
            "average_faithfulness": float(avg_faithfulness) if not np.isnan(avg_
            "average_sufficiency": float(avg_sufficiency) if not np.isnan(avg_su
            "average_comprehensiveness": float(avg_comprehensiveness) if not np.
            "num_examples": int(n if n is not None else total),
        },
        "model_performance": {
            "accuracy": float(accuracy),
            "avg_confidence": float(avg_confidence),
            "correct": int(correct),
            "total": int(total),
        },
    }

    # (Optional) include per-example if you want it in the JSON
    # if per_example is not None:
    #     final_results["per_example"] = [
    #         {
    #             "id": int(r.get("id", i)),
    #             "faithfulness": safe_float(r.get("faithfulness")),
    #             "sufficiency": safe_float(r.get("sufficiency")),
    #             "comprehensiveness": safe_float(r.get("comprehensiveness")),
    #         } for i, r in enumerate(per_example)
    #     ]

    out_path = os.path.join(dirs["LIME_OUTPUT_DIR"], "complete_evaluation_result
    write_json(out_path, final_results)  # uses your NumPy-safe writer
    print(f"✅ Complete results saved to: {out_path}")
    return final_results
```

In [18]:

```
def export_summary_csv(lime_results, metrics_results, dirs):
    import pandas as pd
    m = {r["id"]: r for r in metrics_results} if metrics_results else {}
    rows = []
    for r in lime_results:
        mid = r["id"]
        rows.append({
            "id": mid,
            "true": r["true_label_name"],
            "pred": r["predicted_class"],
            "conf": r["confidence"],
            "lime_score": r.get("lime_score", np.nan),
            "lime_runtime_sec": r.get("lime_runtime_sec", np.nan),
            "faithfulness": m.get(mid, {}).get("faithfulness", np.nan),
            "sufficiency": m.get(mid, {}).get("sufficiency", np.nan),
            "comprehensiveness": m.get(mid, {}).get("comprehensiveness", np.nan)
        })
    df = pd.DataFrame(rows)
    outp = os.path.join(dirs['LIME_OUTPUT_DIR'], "summary.csv")
```

```
    df.to_csv(outp, index=False, encoding="utf-8")
    print(f"📄 Exported CSV: {outp}")
```

MAIN EXECUTION PIPELINE

In [19]:
```
def main(num_examples=50, lime_num_samples=1000, chunk_size=64, force_rebuild=Fa
    """Enhanced main pipeline with all requested features"""
    print("🎯 Starting Enhanced RoBERTa-MNLI + LIME Pipeline for SNLI")
    print("🔧 New features: reproducibility, sanity checks, error analysis, stab

    # Set reproducible seeds
    set_all_seeds(42)

    # Clear GPU memory
    if torch.cuda.is_available():
        print("🔒 Clearing GPU memory...")
        torch.cuda.empty_cache()
        torch.cuda.ipc_collect()
        print("✅ GPU memory cleared.")
    else:
        print("❗ No GPU available; skipping memory cleanup.")

    try:
        # Step 1: Setup
        dirs = setup_directories()
        print("✅ Directories setup complete")

        # Save run configuration
        config = save_run_config(dirs, num_samples=num_examples,
                                 lime_num_samples=lime_num_samples, chunk_size=ch

        # Step 2: Load and sample data
        snli_data = load_snli_dataset_fixed(dirs)
        snli_sample = sample_snli_dataset_fixed(snli_data, dirs, num_samples=num
        processed_snli = preprocess_snli_for_roberta(snli_sample, dirs)

        # Step 3: Load RoBERTa model with unit tests
        classifier = RoBERTaMNLIClassifier("roberta-large-mnli", use_fp16=True)

        # Step 4: Generate LIME explanations
        lime_results = generate_lime_explanations(classifier, processed_snli, di
                                                  num_examples=num_examples,
                                                  lime_num_samples=lime_num_sampl
                                                  chunk_size=chunk_size,
                                                  force_rebuild=force_rebuild)

        # Step 5: Filter stopwords
        filtered_results = filter_stopwords_from_lime(lime_results, dirs)

        # Step 6: Evaluate metrics
        metrics_results = compute_evaluation_metrics(classifier, filtered_result

        export_summary_csv(filtered_results, metrics_results, dirs)

        # Step 7: Sanity checks
        sanity_results = perform_sanity_checks(classifier, filtered_results, dir

        # Step 8: Error analysis
```

```
        error_results = error_analysis(filtered_results, dirs)

        # Step 9: LIME stability sweep
        stability_results = lime_stability_sweep(classifier, processed_snli, dir

        # Step 10: Enhanced analysis with plots
        final_results = analyze_results_enhanced(filtered_results, metrics_resul

        # Step 11: Create comprehensive visualizations
        create_comprehensive_plots(filtered_results, metrics_results, dirs)

        plot_runtime_distribution(filtered_results, dirs)
        plot_per_class_top_words(filtered_results, dirs, by="predicted_class", t
        plot_reliability_diagram(filtered_results, dirs)

        # Step 12: Generate report
        report_content = generate_report(filtered_results, metrics_results, dirs

        print("\n🎉 Enhanced pipeline completed successfully!")
        print("📁 Generated files:")
        print(f"   📊 Results: {dirs['LIME_OUTPUT_DIR']}/complete_evaluation_re
        print(f"   🔧 Config: {dirs['LIME_OUTPUT_DIR']}/run_config.json")
        print(f"   🔍 Sanity: {dirs['LIME_OUTPUT_DIR']}/sanity_check_results.js
        print(f"   ❌ Errors: {dirs['LIME_OUTPUT_DIR']}/error_analysis.json")
        print(f"   🔬 Stability: {dirs['LIME_OUTPUT_DIR']}/lime_stability_sweep
        print(f"   📝 Report: {dirs['LIME_OUTPUT_DIR']}/SNLI_LIME_report.md")
        print(f"   📈 Plots: {dirs['LIME_OUTPUT_DIR']}/plots/")

        return final_results

    except Exception as e:
        print(f"❌ Enhanced pipeline failed: {e}")
        import traceback
        traceback.print_exc()
        return None

def main_with_args():
    """Main function with command line arguments"""
    parser = argparse.ArgumentParser(description='Enhanced GRACE LIME Pipeline')
    parser.add_argument('--num-samples', type=int, default=50, help='Number of e
    parser.add_argument('--lime-samples', type=int, default=1000, help='LIME per
    parser.add_argument('--chunk-size', type=int, default=64, help='Batch proces
    parser.add_argument('--force', action='store_true', help='Force rebuild all

    args = parser.parse_args()

    return main(
        num_examples=args.num_samples,
        lime_num_samples=args.lime_samples,
        chunk_size=args.chunk_size,
        force_rebuild=args.force
    )

if __name__ == "__main__":
    # Run with default parameters (can be called with arguments)
    results = main(num_examples=1000, lime_num_samples=700, chunk_size=64, force

    # Alternative: Uncomment to use command line arguments
    # results = main_with_args()
```

```
# USAGE EXAMPLES:
#
# Basic run:
# python grace_enhanced_pipeline.py
#
# With custom parameters:
# python grace_enhanced_pipeline.py --num-samples 100 --lime-samples 1500 --chun
#
# Force rebuild all cached files:
# python grace_enhanced_pipeline.py --force
#
# Speed vs quality trade-off examples:
# python grace_enhanced_pipeline.py --num-samples 20 --lime-samples 500    # Fas
# python grace_enhanced_pipeline.py --num-samples 100 --lime-samples 2000  # Hig
```

```
🎯 Starting Enhanced RoBERTa-MNLI + LIME Pipeline for SNLI
🔧 New features: reproducibility, sanity checks, error analysis, stability swee
p, comprehensive plots
✅ All seeds set to 42
🔒 Clearing GPU memory...
✅ GPU memory cleared.
✅ Directories setup complete
✅ Run config saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project
\data\lime_outputs\run_config.json
📥 Loading SNLI dataset...
Loading from local cache...
✅ SNLI dataset loaded: 550152 training examples
Sample: {'premise': 'A person on a horse jumps over a broken down airplane.', 'hy
pothesis': 'A person is training his horse for a competition.', 'label': 1}
🎯 Need 1000 samples; currently 0. Extending…
✅ Saved 1000 samples to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\projec
t\data\sampled_snli_data\snli_sample.json
🔄 Preprocessing SNLI data for RoBERTa-MNLI...
✅ Saved 1000 processed examples to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Rese
arch\project\data\processed_snli_data\processed_snli.json
🤖 Loading roberta-large-mnli model...
Using device: cuda
✅ Model loaded successfully!
🧪 Testing model...
Test input: 'The cat is sleeping on the couch.' vs 'The cat is awake.'
Probabilities: [0.9956   0.002132 0.00226 ]
Predicted: contradiction (confidence: 0.9956)
✅ Model test passed!
🔍 Generating LIME explanations for 1000 new examples (target=1000)...
   LIME samples: 700, Chunk size: 64
LIME Explanations:   1%|          | 10/1000 [05:12<8:57:09, 32.55s/it]
💾 Checkpoint: saved 10/1000 items
LIME Explanations:   2%|▏         | 20/1000 [11:27<9:17:50, 34.15s/it]
💾 Checkpoint: saved 20/1000 items
LIME Explanations:   3%|▎         | 30/1000 [18:13<10:24:23, 38.62s/it]
💾 Checkpoint: saved 30/1000 items
LIME Explanations:   4%|▍         | 40/1000 [23:55<8:33:35, 32.10s/it]
💾 Checkpoint: saved 40/1000 items
LIME Explanations:   5%|▌         | 50/1000 [31:13<11:18:59, 42.88s/it]
💾 Checkpoint: saved 50/1000 items
LIME Explanations:   6%|▌         | 60/1000 [37:46<10:13:01, 39.13s/it]
💾 Checkpoint: saved 60/1000 items
LIME Explanations:   7%|▋         | 70/1000 [43:38<9:12:58, 35.68s/it]
```

```
💾 Checkpoint: saved 70/1000 items
LIME Explanations:   8%|▉         | 80/1000 [50:06<9:19:47, 36.51s/it]
💾 Checkpoint: saved 80/1000 items
LIME Explanations:   9%|▉         | 90/1000 [56:37<9:17:12, 36.74s/it]
💾 Checkpoint: saved 90/1000 items
LIME Explanations:  10%|▉         | 100/1000 [1:02:12<8:58:50, 35.92s/it]
💾 Checkpoint: saved 100/1000 items
LIME Explanations:  11%|▉         | 110/1000 [1:08:27<8:46:05, 35.47s/it]
💾 Checkpoint: saved 110/1000 items
LIME Explanations:  12%|▉         | 120/1000 [1:13:46<7:19:18, 29.95s/it]
💾 Checkpoint: saved 120/1000 items
LIME Explanations:  13%|▉         | 130/1000 [1:20:23<8:49:20, 36.51s/it]
💾 Checkpoint: saved 130/1000 items
LIME Explanations:  14%|▉         | 140/1000 [1:26:31<8:30:25, 35.61s/it]
💾 Checkpoint: saved 140/1000 items
LIME Explanations:  15%|▉         | 150/1000 [1:33:01<9:36:06, 40.67s/it]
💾 Checkpoint: saved 150/1000 items
LIME Explanations:  16%|▉         | 160/1000 [1:39:24<10:32:24, 45.17s/it]
💾 Checkpoint: saved 160/1000 items
LIME Explanations:  17%|▉         | 170/1000 [1:45:16<8:31:03, 36.94s/it]
💾 Checkpoint: saved 170/1000 items
LIME Explanations:  18%|▉         | 180/1000 [1:51:07<7:45:38, 34.07s/it]
💾 Checkpoint: saved 180/1000 items
LIME Explanations:  19%|▉         | 190/1000 [1:57:43<8:55:56, 39.70s/it]
💾 Checkpoint: saved 190/1000 items
LIME Explanations:  20%|▉         | 200/1000 [2:04:26<10:53:24, 49.01s/it]
💾 Checkpoint: saved 200/1000 items
LIME Explanations:  21%|▉         | 210/1000 [2:11:26<9:45:02, 44.43s/it]
💾 Checkpoint: saved 210/1000 items
LIME Explanations:  22%|▉         | 220/1000 [2:17:47<8:14:32, 38.04s/it]
💾 Checkpoint: saved 220/1000 items
LIME Explanations:  23%|▉         | 230/1000 [2:23:51<8:05:44, 37.85s/it]
💾 Checkpoint: saved 230/1000 items
LIME Explanations:  24%|▉         | 240/1000 [2:31:38<8:44:22, 41.40s/it]
💾 Checkpoint: saved 240/1000 items
LIME Explanations:  25%|▉         | 250/1000 [2:37:48<7:58:40, 38.29s/it]
💾 Checkpoint: saved 250/1000 items
LIME Explanations:  26%|▉         | 260/1000 [2:44:02<7:02:34, 34.26s/it]
💾 Checkpoint: saved 260/1000 items
LIME Explanations:  27%|▉         | 270/1000 [2:50:23<7:09:50, 35.33s/it]
💾 Checkpoint: saved 270/1000 items
LIME Explanations:  28%|▉         | 280/1000 [2:56:46<7:38:14, 38.19s/it]
💾 Checkpoint: saved 280/1000 items
LIME Explanations:  29%|▉         | 290/1000 [3:03:05<6:49:39, 34.62s/it]
💾 Checkpoint: saved 290/1000 items
LIME Explanations:  30%|▉         | 300/1000 [3:09:05<6:45:13, 34.73s/it]
💾 Checkpoint: saved 300/1000 items
LIME Explanations:  31%|▉         | 310/1000 [3:15:00<6:32:57, 34.17s/it]
💾 Checkpoint: saved 310/1000 items
LIME Explanations:  32%|▉         | 320/1000 [3:22:01<9:56:46, 52.66s/it]
💾 Checkpoint: saved 320/1000 items
LIME Explanations:  33%|▉         | 330/1000 [3:28:30<6:51:56, 36.89s/it]
```

```
💾 Checkpoint: saved 330/1000 items
LIME Explanations:  34%|███       | 340/1000 [3:35:03<8:22:19, 45.67s/it]
💾 Checkpoint: saved 340/1000 items
LIME Explanations:  35%|███       | 350/1000 [3:40:46<6:22:06, 35.27s/it]
💾 Checkpoint: saved 350/1000 items
LIME Explanations:  36%|███       | 360/1000 [3:46:28<6:03:51, 34.11s/it]
💾 Checkpoint: saved 360/1000 items
LIME Explanations:  37%|███▋      | 370/1000 [3:52:21<6:25:17, 36.69s/it]
💾 Checkpoint: saved 370/1000 items
LIME Explanations:  38%|███▊      | 380/1000 [3:58:00<5:44:38, 33.35s/it]
💾 Checkpoint: saved 380/1000 items
LIME Explanations:  39%|███▉      | 390/1000 [4:04:09<6:14:33, 36.84s/it]
💾 Checkpoint: saved 390/1000 items
LIME Explanations:  40%|████      | 400/1000 [4:10:41<6:46:24, 40.64s/it]
💾 Checkpoint: saved 400/1000 items
LIME Explanations:  41%|████      | 410/1000 [4:17:02<6:22:37, 38.91s/it]
💾 Checkpoint: saved 410/1000 items
LIME Explanations:  42%|████▏     | 420/1000 [4:23:19<6:00:42, 37.32s/it]
💾 Checkpoint: saved 420/1000 items
LIME Explanations:  43%|████▎     | 430/1000 [4:29:26<6:00:39, 37.96s/it]
💾 Checkpoint: saved 430/1000 items
LIME Explanations:  44%|████▍     | 440/1000 [4:35:42<6:14:38, 40.14s/it]
💾 Checkpoint: saved 440/1000 items
LIME Explanations:  45%|████▌     | 450/1000 [4:42:29<6:01:35, 39.45s/it]
💾 Checkpoint: saved 450/1000 items
LIME Explanations:  46%|████▌     | 460/1000 [4:48:04<5:10:47, 34.53s/it]
💾 Checkpoint: saved 460/1000 items
LIME Explanations:  47%|████▋     | 470/1000 [4:54:39<6:21:32, 43.19s/it]
💾 Checkpoint: saved 470/1000 items
LIME Explanations:  48%|████▊     | 480/1000 [5:00:48<5:35:21, 38.70s/it]
💾 Checkpoint: saved 480/1000 items
LIME Explanations:  49%|████▉     | 490/1000 [5:07:05<5:22:43, 37.97s/it]
💾 Checkpoint: saved 490/1000 items
LIME Explanations:  50%|█████     | 500/1000 [5:14:08<5:42:38, 41.12s/it]
💾 Checkpoint: saved 500/1000 items
LIME Explanations:  51%|█████     | 510/1000 [5:20:42<5:42:28, 41.94s/it]
💾 Checkpoint: saved 510/1000 items
LIME Explanations:  52%|█████▏    | 520/1000 [5:26:52<5:08:45, 38.60s/it]
💾 Checkpoint: saved 520/1000 items
LIME Explanations:  53%|█████▎    | 530/1000 [5:33:05<5:11:12, 39.73s/it]
💾 Checkpoint: saved 530/1000 items
LIME Explanations:  54%|█████▍    | 540/1000 [5:39:21<5:07:43, 40.14s/it]
💾 Checkpoint: saved 540/1000 items
LIME Explanations:  55%|█████▌    | 550/1000 [5:45:34<4:34:09, 36.55s/it]
💾 Checkpoint: saved 550/1000 items
LIME Explanations:  56%|█████▌    | 560/1000 [5:50:38<3:31:56, 28.90s/it]
💾 Checkpoint: saved 560/1000 items
LIME Explanations:  57%|█████▋    | 570/1000 [5:56:28<4:33:09, 38.11s/it]
💾 Checkpoint: saved 570/1000 items
LIME Explanations:  58%|█████▊    | 580/1000 [6:02:53<5:10:52, 44.41s/it]
💾 Checkpoint: saved 580/1000 items
LIME Explanations:  59%|█████▉    | 590/1000 [6:09:17<4:21:42, 38.30s/it]
```

```
💾 Checkpoint: saved 590/1000 items
LIME Explanations:  60%|██████    | 600/1000 [6:15:30<3:45:23, 33.81s/it]
💾 Checkpoint: saved 600/1000 items
LIME Explanations:  61%|██████    | 610/1000 [6:22:01<3:58:14, 36.65s/it]
💾 Checkpoint: saved 610/1000 items
LIME Explanations:  62%|██████    | 620/1000 [6:28:10<3:53:03, 36.80s/it]
💾 Checkpoint: saved 620/1000 items
LIME Explanations:  63%|██████    | 630/1000 [6:34:02<3:29:43, 34.01s/it]
💾 Checkpoint: saved 630/1000 items
LIME Explanations:  64%|██████    | 640/1000 [6:39:49<3:47:14, 37.87s/it]
💾 Checkpoint: saved 640/1000 items
LIME Explanations:  65%|██████    | 650/1000 [6:46:10<3:30:05, 36.01s/it]
💾 Checkpoint: saved 650/1000 items
LIME Explanations:  66%|██████    | 660/1000 [6:52:09<3:41:16, 39.05s/it]
💾 Checkpoint: saved 660/1000 items
LIME Explanations:  67%|██████    | 670/1000 [6:59:26<4:01:31, 43.91s/it]
💾 Checkpoint: saved 670/1000 items
LIME Explanations:  68%|██████    | 680/1000 [7:05:16<3:17:39, 37.06s/it]
💾 Checkpoint: saved 680/1000 items
LIME Explanations:  69%|██████    | 690/1000 [7:11:15<2:58:46, 34.60s/it]
💾 Checkpoint: saved 690/1000 items
LIME Explanations:  70%|██████    | 700/1000 [7:17:15<2:53:49, 34.77s/it]
💾 Checkpoint: saved 700/1000 items
LIME Explanations:  71%|██████    | 710/1000 [7:23:35<3:15:48, 40.51s/it]
💾 Checkpoint: saved 710/1000 items
LIME Explanations:  72%|██████    | 720/1000 [7:30:38<3:21:09, 43.10s/it]
💾 Checkpoint: saved 720/1000 items
LIME Explanations:  73%|██████    | 730/1000 [7:36:31<2:56:31, 39.23s/it]
💾 Checkpoint: saved 730/1000 items
LIME Explanations:  74%|██████    | 740/1000 [7:42:56<2:41:20, 37.23s/it]
💾 Checkpoint: saved 740/1000 items
LIME Explanations:  75%|██████    | 750/1000 [7:48:46<2:21:35, 33.98s/it]
💾 Checkpoint: saved 750/1000 items
LIME Explanations:  76%|██████    | 760/1000 [7:55:01<2:32:34, 38.15s/it]
💾 Checkpoint: saved 760/1000 items
LIME Explanations:  77%|██████    | 770/1000 [8:01:05<2:06:46, 33.07s/it]
💾 Checkpoint: saved 770/1000 items
LIME Explanations:  78%|██████    | 780/1000 [8:07:36<2:25:20, 39.64s/it]
💾 Checkpoint: saved 780/1000 items
LIME Explanations:  79%|██████    | 790/1000 [8:14:02<2:12:53, 37.97s/it]
💾 Checkpoint: saved 790/1000 items
LIME Explanations:  80%|██████    | 800/1000 [8:20:18<1:59:01, 35.71s/it]
💾 Checkpoint: saved 800/1000 items
LIME Explanations:  81%|██████    | 810/1000 [8:26:20<1:59:59, 37.89s/it]
💾 Checkpoint: saved 810/1000 items
LIME Explanations:  82%|██████    | 820/1000 [8:32:09<1:34:13, 31.41s/it]
💾 Checkpoint: saved 820/1000 items
LIME Explanations:  83%|██████    | 830/1000 [8:37:28<1:21:00, 28.59s/it]
💾 Checkpoint: saved 830/1000 items
LIME Explanations:  84%|██████    | 840/1000 [8:43:33<1:36:54, 36.34s/it]
💾 Checkpoint: saved 840/1000 items
LIME Explanations:  85%|██████    | 850/1000 [8:49:36<1:25:18, 34.12s/it]
```

```
💾 Checkpoint: saved 850/1000 items
LIME Explanations:  86%|████████▌ | 860/1000 [8:55:40<1:30:35, 38.83s/it]
💾 Checkpoint: saved 860/1000 items
LIME Explanations:  87%|████████▋ | 870/1000 [9:01:28<1:26:49, 40.07s/it]
💾 Checkpoint: saved 870/1000 items
LIME Explanations:  88%|████████▊ | 880/1000 [9:08:13<1:23:08, 41.57s/it]
💾 Checkpoint: saved 880/1000 items
LIME Explanations:  89%|████████▉ | 890/1000 [9:14:07<1:08:05, 37.14s/it]
💾 Checkpoint: saved 890/1000 items
LIME Explanations:  90%|█████████ | 900/1000 [9:19:11<49:46, 29.87s/it]
💾 Checkpoint: saved 900/1000 items
LIME Explanations:  91%|█████████ | 910/1000 [9:24:44<50:22, 33.58s/it]
💾 Checkpoint: saved 910/1000 items
LIME Explanations:  92%|█████████▏ | 920/1000 [9:30:47<57:01, 42.77s/it]
💾 Checkpoint: saved 920/1000 items
LIME Explanations:  93%|█████████▎ | 930/1000 [9:36:14<44:16, 37.95s/it]
💾 Checkpoint: saved 930/1000 items
LIME Explanations:  94%|█████████▍ | 940/1000 [9:42:07<36:48, 36.82s/it]
💾 Checkpoint: saved 940/1000 items
LIME Explanations:  95%|█████████▌ | 950/1000 [9:48:03<27:49, 33.39s/it]
💾 Checkpoint: saved 950/1000 items
LIME Explanations:  96%|█████████▌ | 960/1000 [9:54:08<26:26, 39.65s/it]
💾 Checkpoint: saved 960/1000 items
LIME Explanations:  97%|█████████▋ | 970/1000 [10:00:40<20:14, 40.47s/it]
💾 Checkpoint: saved 970/1000 items
LIME Explanations:  98%|█████████▊ | 980/1000 [10:06:25<11:17, 33.87s/it]
💾 Checkpoint: saved 980/1000 items
LIME Explanations:  99%|█████████▉| 990/1000 [10:12:04<06:06, 36.61s/it]
💾 Checkpoint: saved 990/1000 items
LIME Explanations: 100%|██████████| 1000/1000 [10:18:02<00:00, 37.08s/it]
💾 Checkpoint: saved 1000/1000 items
✅ Generated 1000 new LIME explanations (total=1000)
✅ Saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_explanations_roberta.json

🧹 Filtering stopwords from LIME attributions...
✅ Filtered results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_explanations_filtered.json
Average attribution reduction: 4.2 tokens
📊 Computing evaluation metrics (k=3)...
Computing metrics: 100%|██████████| 1000/1000 [06:03<00:00,  2.75it/s]
📄 Exported CSV: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\summary.csv
🔍 Performing sanity checks...
Found 861 correct, 139 incorrect predictions
🔀 Running shuffle sanity test...
✅ Sanity check results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\sanity_check_results.json
🔍 Performing error analysis...
✅ Error analysis saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\error_analysis.json
🔬 Running LIME stability mini-sweep...
Stability sweep: 100%|██████████| 20/20 [10:38<00:00, 31.90s/ex, ns=1000, kw=default, ex 10/10]
```

```
✅ Stability sweep results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\lime_stability_sweep.json
📈 Enhanced results analysis...

🎯 FINAL EVALUATION RESULTS:
   Model Accuracy: 0.290
   Average Confidence: 0.934
   Average Faithfulness: 0.102 ± 0.137
   Average Sufficiency: 0.675 ± 0.318
   Average Comprehensiveness: 0.063 ± 0.159
✅ Complete results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\complete_evaluation_results.json
📊 Creating comprehensive visualization plots...
❌ Enhanced pipeline failed: cannot access local variable 'label_names' where it is not associated with a value
Traceback (most recent call last):
  File "C:\Users\Work\AppData\Local\Temp\ipykernel_18060\674430789.py", line 63, in main
    create_comprehensive_plots(filtered_results, metrics_results, dirs)
  File "C:\Users\Work\AppData\Local\Temp\ipykernel_18060\303305997.py", line 53, in create_comprehensive_plots
    cm = confusion_matrix(true_labels_numeric, pred_labels_numeric, labels=label_names)
                                                                           ^^^^^^
^^^^^
UnboundLocalError: cannot access local variable 'label_names' where it is not associated with a value
```

In [20]:

```python
# ==== RESUME: reload saved artifacts & set up classifier ====
from pathlib import Path
import os, json, numpy as np

dirs = setup_directories()  # uses your existing helper

def _read_any(path):
    with open(path, "r", encoding="utf-8") as f:
        obj = json.load(f)
    return obj.get("data", obj) if isinstance(obj, dict) else obj

LIME_DIR = Path(dirs["LIME_OUTPUT_DIR"])
lime_path = LIME_DIR / "lime_explanations_roberta.json"
filtered_path = LIME_DIR / "lime_explanations_filtered.json"
metrics_partial_path = LIME_DIR / "metrics_partial.json"

# 1) LIME results (from disk; don't recompute)
assert lime_path.exists(), f"Missing {lime_path} — explanations must exist alrea
lime_results = _read_any(lime_path)
print(f"✅ Loaded LIME explanations: {len(lime_results)}")

# 2) Filtered results if already saved; otherwise try to compute quickly
if filtered_path.exists():
    filtered_results = _read_any(filtered_path)
    print(f"✅ Loaded filtered results: {len(filtered_results)}")
else:
    # fall back to filter function if present; else just use raw
    if "filter_stopwords_from_lime" in globals():
        filtered_results = filter_stopwords_from_lime(lime_results, dirs)
    elif "filter_stopwords_from_LIME" in globals():
        filtered_results = filter_stopwords_from_LIME(lime_results, dirs)
    else:
```

```
        print("⚠️ Filter function not found; using raw LIME results.")
        filtered_results = lime_results

# 3) Classifier (reuse if already in memory)
if "classifier" in globals():
    clf = classifier
elif "clf" in globals():
    clf = clf
else:
    clf = RoBERTaMNLIClassifier("roberta-large-mnli", use_fp16=True)

# 4) Processed SNLI (for safety, in case later steps need it)
if "processed_snli" not in globals():
    processed_json = Path(dirs["PROCESSED_SNLI_DIR"]) / "processed_snli.json"
    if processed_json.exists():
        processed_snli = _read_any(processed_json)
        print(f"📄 Loaded processed SNLI: {len(processed_snli)}")
    else:
        print("ℹ️ processed_snli not needed right now (continuing).")

# 5) Quick status
print("📊 Ready to resume metrics/analysis/plots.")
```

```
✅ Loaded LIME explanations: 1000
✅ Loaded filtered results: 1000
🤖 Loading roberta-large-mnli model...
Using device: cuda
✅ Model loaded successfully!
🧪 Testing model...
Test input: 'The cat is sleeping on the couch.' vs 'The cat is awake.'
Probabilities: [0.9956   0.002132 0.00226 ]
Predicted: contradiction (confidence: 0.9956)
✅ Model test passed!
📄 Loaded processed SNLI: 1000
📊 Ready to resume metrics/analysis/plots.
```

In [21]:

```
# ==== RESUME: metrics (append), analysis, plots, snapshot ====
# This will pick up from metrics_partial.json if present.
try:
    metrics_results = compute_evaluation_metrics(
        clf, filtered_results, k=3, dirs=dirs, save_every=10
    )
    print(f"✅ Metrics computed/resumed: {len(metrics_results)} items")
except Exception as e:
    print("❌ Metrics step failed:", e)
    raise

# Analysis (pick enhanced if available)
try:
    if "analyze_results_enhanced" in globals():
        final_results = analyze_results_enhanced(filtered_results, metrics_resul
    else:
        final_results = analyze_results(filtered_results, metrics_results, dirs)
    print("✅ Analysis complete.")
except Exception as e:
    print("❌ Analysis step failed:", e)
    raise

# Plots (safe to call multiple times; they overwrite files)
try:
```

```
    if "create_comprehensive_plots" in globals():
        create_comprehensive_plots(filtered_results, metrics_results, dirs)
    # optional extras if you added them:
    if "plot_runtime_distribution" in globals():
        plot_runtime_distribution(filtered_results, dirs)
    if "plot_per_class_top_words" in globals():
        plot_per_class_top_words(filtered_results, dirs, by="predicted_class", t
    if "plot_reliability_diagram" in globals():
        plot_reliability_diagram(filtered_results, dirs)
    print("🖼️ Plots saved.")
except Exception as e:
    print("⚠️ Plotting step had an issue:", e)

# CSV export (if you added the helper; otherwise skip)
if "export_summary_csv" in globals():
    try:
        export_summary_csv(filtered_results, metrics_results, dirs)
    except Exception as e:
        print("⚠️ CSV export issue:", e)

# Final snapshot so you can see what got produced without opening files
snapshot = {
    "schema_version": SCHEMA_VERSION if "SCHEMA_VERSION" in globals() else "unkn
    "counts": {
        "lime": len(lime_results),
        "filtered": len(filtered_results),
        "metrics": len(metrics_results) if isinstance(metrics_results, list) els
    },
    "paths": {
        "lime": str(lime_path),
        "filtered": str(filtered_path),
        "metrics_partial": str(metrics_partial_path),
        "complete_results": str(LIME_DIR / "complete_evaluation_results.json"),
        "plots_dir": str(LIME_DIR / "plots")
    }
}
print("📌 Snapshot:", snapshot)
```

```
📊 Computing evaluation metrics (k=3)...
⏩ Resuming metrics from 999 items.
Computing metrics: 100%|██████████| 1000/1000 [00:00<00:00, 502974.46it/s]
✅ Metrics computed/resumed: 999 items
📈 Enhanced results analysis...
```

```
🎯 FINAL EVALUATION RESULTS:
   Model Accuracy: 0.290
   Average Confidence: 0.934
   Average Faithfulness: 0.102 ± 0.137
   Average Sufficiency: 0.675 ± 0.318
   Average Comprehensiveness: 0.063 ± 0.159
✅ Complete results saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\complete_evaluation_results.json
✅ Analysis complete.
📊 Creating comprehensive visualization plots...
⚠️ Plotting step had an issue: cannot access local variable 'label_names' where it is not associated with a value
📄 Exported CSV: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\summary.csv
📌 Snapshot: {'schema_version': '1.1', 'counts': {'lime': 1000, 'filtered': 1000, 'metrics': 999}, 'paths': {'lime': 'c:\\Users\\Work\\OneDrive\\Desktop\\ChatGPT\\Research\\project\\data\\lime_outputs\\lime_explanations_roberta.json', 'filtered': 'c:\\Users\\Work\\OneDrive\\Desktop\\ChatGPT\\Research\\project\\data\\lime_outputs\\lime_explanations_filtered.json', 'metrics_partial': 'c:\\Users\\Work\\OneDrive\\Desktop\\ChatGPT\\Research\\project\\data\\lime_outputs\\metrics_partial.json', 'complete_results': 'c:\\Users\\Work\\OneDrive\\Desktop\\ChatGPT\\Research\\project\\data\\lime_outputs\\complete_evaluation_results.json', 'plots_dir': 'c:\\Users\\Work\\OneDrive\\Desktop\\ChatGPT\\Research\\project\\data\\lime_outputs\\plots'}}
```

In [22]:

```python
from pathlib import Path
from IPython.display import display, Image
import os, json

dirs = setup_directories()  # if already defined, this is a no-op
plots_dir = Path(dirs["LIME_OUTPUT_DIR"]) / "plots"
assert plots_dir.exists(), f"No plots directory found at {plots_dir}"

pngs = sorted(plots_dir.glob("*.png"))
svgs = sorted(plots_dir.glob("*.svg"))

print(f"Found {len(pngs)} PNG(s) and {len(svgs)} SVG(s) in {plots_dir}")
for p in pngs:
    print("Showing:", p.name)
    display(Image(filename=str(p)))

# If you also saved SVGs and want to view them inline:
try:
    from IPython.display import SVG
    for s in svgs:
        print("Showing:", s.name)
        display(SVG(filename=str(s)))
except Exception as e:
    print("SVG display not available:", e)
```

```
Found 0 PNG(s) and 0 SVG(s) in c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\lime_outputs\plots
```

In [25]:

```python
# Rebuild plots only (idempotent)
lime_path = Path(dirs["LIME_OUTPUT_DIR"]) / "lime_explanations_roberta.json"
filtered_path = Path(dirs["LIME_OUTPUT_DIR"]) / "lime_explanations_filtered.json

def _read_any(p):
    obj = json.load(open(p, "r", encoding="utf-8"))
    return obj.get("data", obj) if isinstance(obj, dict) else obj
```

```
lime_results = _read_any(lime_path)
filtered_results = _read_any(filtered_path) if filtered_path.exists() else lime_

# Recreate plots; these functions only read, they won't redo explanations
if "create_comprehensive_plots" in globals():
    create_comprehensive_plots(filtered_results, None, dirs)
if "plot_runtime_distribution" in globals():
    plot_runtime_distribution(filtered_results, dirs)
if "plot_per_class_top_words" in globals():
    plot_per_class_top_words(filtered_results, dirs, by="predicted_class", top_n
if "plot_reliability_diagram" in globals():
    plot_reliability_diagram(filtered_results, dirs)
```

```
📊 Creating comprehensive visualization plots...
✅ All plots saved to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\d
ata\lime_outputs\plots
📈 Plotting LIME runtime distribution...
📈 Plotting top words per class...
📈 Plotting reliability diagram...
```